Bachelor Thesis

April 16, 2026

Attribute–Embedding Correlation

A distribution divergence approach on Filtered Vector Search

Matteo Iulian Adam

of University of Zurich, Switzerland (23-744-519)

supervised by

Prof. Dr. Yannis Chronis, ETH
Prof. Dr. Dan Olteanu, UZH
Dr. Mageirakos Vasilis, ETH

University of Zurich[UZH]

Bachelor Thesis

Attribute–Embedding Correlation

A distribution divergence approach on Filtered Vector Search

Matteo Iulian Adam

University of Zurich[UZH]

Bachelor Thesis

Author: Matteo Iulian Adam, matteoiulian.adam@uzh.ch

URL: https://github.com/johndonneUZH/thesis

Project period: 2026-01-15 - 2026-07-01

Systems Group, Institute for Computing Platforms
Department of Computer Science (D-INFK), ETH Zurich

Acknowledgements

Abstract

Contents

1 Introduction **1**
1.1 Motivation and Problem Setting . . . 1
1.2 Research Gap . . . 1
1.3 Research Objectives & Questions . . . 1
1.4 Thesis Structure . . . 2

2 Background and related work **3**
2.1 Vector Search . . . 3
2.1.1 Vector representation . . . 3
2.1.2 Nearest Neighbor Search . . . 3
2.1.3 Approximate Nearest Neighbor Search . . . 3
2.1.4 Metrics for ANNS Evaluation . . . 4
2.2 Filtered Vector Search . . . 5
2.2.1 Selectivity . . . 5
2.2.2 Correlation . . . 6
2.3 Execution Strategies . . . 7
2.3.1 Pre-filtering . . . 7
2.3.2 Post-filtering . . . 7
2.3.3 Inline Filtering . . . 7

3 Methodology **9**
3.1 Data Model . . . 9
3.2 Query Model . . . 9
3.3 Evaluation Setup . . . 9
3.4 Objective . . . 9

4 Dataset Construction **11**
4.1 Design Goals . . . 11
4.2 Correlated Attribute Generation . . . 11
4.2.1 Clustering . . . 11
4.2.2 Cluster Anchors . . . 11
4.2.3 Gaussian Weighting . . . 12
4.2.4 Bandwidth Parameter . . . 13
4.2.5 Soft Assignment . . . 13
4.2.6 Normalization and Quantization . . . 14
4.3 Uncorrelated Attribute Generation . . . 14
4.4 Query Range Generation . . . 14
4.4.1 Nearest-Neighbor Attribute Summary . . . 14
4.4.2 Positive Range Construction . . . 15
4.4.3 Negative Range Construction . . . 15
4.4.4 Random Range Construction . . . 15
4.4.5 Selectivity Control . . . 16
4.5 Dataset Validation . . . 16
4.5.1 Neighbor Consistency and Attribute Distribution . . . 16

4.5.2 PCA Visualization . . . 17

5 Correlation Metric **19**
5.1 Precomputed statistics . . . 19
5.1.1 Cluster centroids. . . . 19
5.1.2 Attribute distributions. . . . 19
5.1.3 Query-to-cluster mapping. . . . 21
5.2 Approximation quality . . . 22
5.2.1 Bounding the approximation error . . . 22
5.2.2 Error decomposition . . . 23
5.2.2.1 Quantization error (Geometric Discretization). . . . 23
5.2.2.2 Estimation error (Statistical Estimation). . . . 24
5.2.3 Bias–variance trade-off . . . 25
5.2.3.1 Effect of the number of clusters k. . . . 25
5.2.3.2 Effect of radius expansion λ. . . . 25

6 System & Execution Strategies **27**
6.1 Pre-filtering . . . 27
6.2 Post-filtering . . . 27
6.3 Inline Filtering . . . 27
6.4 Implementation Details . . . 27

7 Experimental Setup **29**
7.1 Offline computed statistics . . . 29
7.2 Evaluation Metrics . . . 29
7.3 Ground Truth Computation . . . 29

8 Results & Discussion **31**

Bibliography **33**

A First Appendix **35**

B Second Appendix **37**

1. Introduction

1.1 Motivation and Problem Setting Consider a user searching for a shirt similar to a high-end item, but with a price below a given threshold (e.g., under 30). This query combines two components: similarity, to find visually or stylistically related items, and filtering, to enforce a constraint on structured attributes such as price.

This type of query is representative of modern applications, where vector search is combined with structured predicates. In such settings, the goal is not only to retrieve similar items, but to do so within a restricted subset of the data.

Processing these queries efficiently is challenging. While vector search techniques exploit the geometric structure of the embedding space, filtering restricts the search to a subset of the data. This restriction not only reduces the number of candidate points, but can also change their distribution in the embedding space.

A key factor is how the filtered vectors are positioned relative to the query. In some cases, the filter selects vectors that are concentrated near the query, making them easy to retrieve. In other cases, they are located farther away, requiring the search to explore less relevant regions of the space.

This difference reflects how structured attributes interact with the embedding space. When attributes and embeddings are aligned, filtering preserves the local neighborhood of the query. When they are not aligned, filtering disrupts this structure and makes the search more difficult.

Capturing this effect is essential for efficient query processing, as it directly influences how different execution strategies behave and how much work is required to retrieve relevant results.

1.2 Research Gap Existing work, however, typically handles this interaction only indirectly and in a system-specific manner. In particular, most evaluations rely on synthetic workloads where attributes are generated independently of the vector data.

Starting from NHQ [1], prior work constructs such workloads by attaching synthetic attributes to vectors, following approaches used in systems such as Milvus [2] and Multiattribute ANNS [3]. In these settings, attribute values are assigned by randomly sampling from predefined categories. While this enables controlled experiments, it ignores any connection between attributes and embeddings. Similarly, Filtered-DiskANN [4] generates labels independently of the vector space, explicitly removing any correlation.

As a result, current approaches do not capture how filters and vectors interact in practice, and this effect remains largely unmodeled.

1.3 Research Objectives & Questions The main objective is to treat this relationship as a property of the data and to define a quantitative measure that captures how filter predicates relate to the vector distribution. This enables a more systematic understanding of how filtering affects nearest neighbor search.

This leads to the following research questions:

- How can the relationship between filter predicates and the vector space be formally defined and measured?

- How does this relationship affect the behavior and performance of hybrid search methods?
- How can this information be used to improve query processing beyond assumptions of independence?

The goal is to develop a general and system-independent framework that can be applied across different index structures and query execution methods, including both exact and approximate search.

1.4 Thesis Structure The remainder of this thesis is organized as follows.

Chapter 2 provides the necessary background and reviews related work. It introduces vector search, approximate nearest neighbor methods, and filtered vector search, including key concepts such as selectivity, correlation, and common execution strategies.

Chapter 3 formalizes the problem setting by defining the data model, query model, and the objective of the study.

Chapter 4 describes the construction of the datasets used in this thesis. In particular, it explains how both correlated and uncorrelated attributes are generated, as well as how query ranges and selectivity are controlled.

Chapter 5 introduces the proposed correlation metric. It presents the comparison between local and global attribute distributions, the use of Jensen–Shannon divergence, and the incorporation of lift to capture query-dependent effects.

Chapter 6 describes the system setup and the execution strategies considered, including pre-filtering, post-filtering, and inline filtering, along with implementation details.

Chapter 7 outlines the experimental setup, including evaluation metrics and the computation of ground truth.

2. Background and related work

2.1 Vector Search

2.1.1 Vector representation The representation of complex items (such as text, images, or user behavior) as high-dimensional vectors has become increasingly important in modern data-intensive applications. These vectors, commonly referred to as embeddings, are typically learnt using machine learning models and capture semantic or structural relationships in the data [5]. As a result, semantically similar items are mapped to nearby points in a continuous vector space, enabling similarity to be measured using geometric distance functions such as cosine similarity or Euclidean distance.

The widespread adoption of this paradigm has led to the emergence of vector databases, which extend traditional database systems by enabling efficient similarity-based retrieval. Unlike conventional databases that operate on structured attributes using exact or range-based predicates, vector databases are designed to efficiently retrieve similar items from large collections of vectors by leveraging specialized index structures (e.g., graph-based or tree-based indexes) and approximate nearest neighbor search algorithms [6].

2.1.2 Nearest Neighbor Search Given a dataset of vectors $\{x_1, \ldots, x_n\} \subset \mathbb{R}^d$, the nearest neighbor search problem consists of identifying, for a query vector $q \in \mathbb{R}^d$, the vectors in the dataset that are most similar to q. This problem is also known as the *post office problem*, where one seeks the closest facility to a given location. In the k-nearest neighbor (kNN) setting, the goal is to retrieve the k most similar vectors.

To formalize this notion of similarity, we use a function $D(\cdot, \cdot)$ that assigns a numerical score to a pair of vectors, reflecting how far apart they are in the underlying space. Common choices include Euclidean distance, cosine similarity, and inner product-based measures, each capturing different notions of similarity and directly influencing retrieval quality.

Given such a function, a natural solution is to compute $D(q, x_i)$ for every vector x_i in the dataset and return the closest ones. This approach guarantees exact results, but requires scanning the entire dataset and performing a computation of cost $\mathcal{O}(d)$ per vector, resulting in a total time complexity of $\mathcal{O}(n \cdot d)$. As both the number of vectors n and the dimensionality d grow, this linear scan becomes computationally expensive and leads to high query latency [7]. Moreover, in high-dimensional spaces, the contrast between nearby and far-away points tends to diminish, making it harder to distinguish meaningful neighbors [8].

These limitations motivate the use of approximate nearest neighbor approaches.

2.1.3 Approximate Nearest Neighbor Search Approximate nearest neighbor (ANN) methods trade exactness for efficiency by exploring only a subset of the dataset while aiming to preserve high recall. Instead of exhaustively comparing a query against all data points, ANN algorithms rely on specialized index structures and heuristics to efficiently identify promising candidates.

A prominent class is based on hashing techniques, such as locality-sensitive hashing (LSH), which project high-dimensional vectors into lower-dimensional hash spaces in a way that preserves similarity [9], [10]. The key idea behind LSH is to use hash functions that increase the probability of collision for similar vectors while reducing it for dissimilar ones. This enables efficient retrieval of candidate neighbors from

hash buckets, often achieving sub-linear query time. The downside is that LSH-based methods may require multiple hash tables and can suffer from reduced accuracy or increased memory usage in high-dimensional settings.

Graph-based indexing methods have demonstrated particularly strong empirical performance. In particular, the Hierarchical Navigable Small World (HNSW) graph arranges data points in a multi-layer structure and uses greedy search to quickly reach nearest neighbors [11]. Due to its strong balance between efficiency and accuracy, HNSW has become widely adopted in modern vector search systems, but it typically requires the entire index to reside in main memory. For larger, disk-resident datasets, approaches such as DiskANN instead rely on a single-layer graph (Vamana) that is better suited for disk-based access, avoiding the multi-layer traversal of HNSW and enabling efficient search at billion scale [12].

Recent work has also explored learning-based approaches, where models are used to guide how the search structure is built or traversed [13], [14]. Instead of relying only on manually designed heuristics, these methods learn from the data how to partition the vector space or which regions are more likely to contain relevant neighbors. This makes the search process more efficient and highlights that ANN performance depends not only on the search algorithm, but also on how the data is organized.

In this thesis, ANN search is implemented using the FAISS library [15]. It provides a unified framework that implements a variety of ANN techniques, including graph-based methods such as HNSW, as well as exact search for ground truth computation.

2.1.4 Metrics for ANNS Evaluation ANN systems are typically evaluated along three main dimensions: retrieval quality, query efficiency, and resource usage, reflecting the trade-offs inherent in approximate search [15], [16].

A primary measure of retrieval quality is *recall@k*, which quantifies how many of the true nearest neighbors are retrieved by the approximate method. Let R_k^* denote the set of the k exact nearest neighbors and R_k the set returned by the ANN method. Then:

$$\text{recall@k} = \frac{|R_k \cap R_k^*|}{|R_k^*|}.$$

More generally, n-recall@k measures how many of the true nearest neighbors are successfully retrieved by the approximate method. Concretely, it is the fraction of the top-n ground truth nearest neighbors that appear among the top-k approximate results, where $n \leq k$ [15]. A common choice is $n = k$, in which case the metric reflects how well the approximate result set matches the exact one.

The definition of recall@k varies across the literature: some works use 1-recall@k, focusing only on the nearest neighbor, while others use k-recall@k, which evaluates the retrieved set as a whole. In this thesis, we use the latter.

Query efficiency is typically measured in terms of latency and throughput (queries per second, QPS). Latency captures the time required to answer a single query, while throughput reflects the number of queries that can be processed per unit time. Together, these metrics determine how well a system scales under different workloads.

Resource usage further influences these metrics, including memory consumption, index construction time, and, in disk-based settings, I/O cost during search [15]. For example, graph-based indexes require additional memory to store connectivity information, while compression techniques reduce memory usage at the cost of lower recall.

2.2 Filtered Vector Search Let (X, d) be a metric space, where $X \subseteq \mathbb{R}^d$ is a finite set of vectors and $d(\cdot, \cdot)$ is a distance function (e.g., Euclidean or cosine distance). Each vector $x \in X$ is associated with structured attributes $a(x) \in \mathcal{A}$, where $\mathcal{A}$ denotes the attribute space.

A *predicate* is a Boolean function

$$p : \mathcal{A} \to \{0, 1\},$$

which defines a subset of vectors

$$X_p := \{x \in X \mid p(a(x)) = 1\}.$$

Given a query vector $q \in \mathbb{R}^d$ and an integer k, the filtered k-nearest neighbor problem is to retrieve the k closest vectors to q among those that satisfy the predicate:

$$R_k(q, p) = \text{top-}k \text{ elements of } X_p \text{ with respect to } d(q, \cdot).$$

The defining characteristic of filtered vector search is that the query is evaluated over the subset X_p, rather than over the full dataset X. That is, the search domain changes from (X, d) to (X_p, d). This restriction does not only reduce the number of candidate points; it changes the distribution and structure of the data on which the search is performed.

Formally, if μ denotes the empirical distribution over X, then restricting to X_p corresponds to conditioning on the predicate:

$$\mu_p(x) = \mathbb{P}\big(x \mid p(a(x)) = 1\big).$$

In general, μ_p can differ significantly from μ. For instance, X_p may be much smaller, unevenly distributed, or composed of points that are far apart in the embedding space.

In the full dataset X, the points closest to q may not satisfy the predicate and be excluded from X_p. Thus, the nearest neighbors of q within X_p can be entirely different from the nearest neighbors in X.

Each predicate p defines therefore a different search problem. The subset X_p can vary in size, distribution, and connectivity, and these variations determine how difficult the search is. As a result, filtered vector search is inherently *query-dependent*: instead of operating on a single fixed dataset, the system must handle many different subsets defined at query time.

This creates a fundamental challenge. Most NN methods are designed under assumptions about the global structure of X. When applied to a subset X_p that does not follow the same structure, these methods may become inefficient or lose accuracy. [1], [17], [18], [19].

2.2.1 Selectivity Selectivity is a fundamental concept in query processing that quantifies the fraction of a dataset satisfying a predicate. Given a dataset X and a predicate p, the selectivity of p is defined as

$$\text{sel}(p) = \frac{|X_p|}{|X|}.$$

The cardinality of p is given by $|X_p|$.

Cardinality estimation has been extensively studied in the database literature due to its central role in query optimization. Classical approaches often assume attribute independence, approximating the selectivity of a conjunction of predicates as the product of their individual selectivities. While computationally convenient, this assumption can lead to large estimation errors in the presence of correlated attributes [20]. Such estimates, combined with statistics such as histograms or samples, are used by query planners to select execution strategies, and inaccuracies can lead to suboptimal plans [21], [22].

In filtered vector search, selectivity determines the size of the candidate set. For high selectivity, $|X_p|$ remains large and the search space closely resembles the full dataset. For low selectivity, $|X_p|$ is small, reducing the number of candidates but restricting the available neighbors.

However, selectivity alone does not fully characterize query difficulty. Two predicates p_1, p_2 with $\text{sel}(p_1) = \text{sel}(p_2)$ may induce subsets X_{p_1} and X_{p_2} with very different structures in the embedding space, leading to different retrieval behavior. In this thesis, we therefore study queries across varying selectivity levels while explicitly accounting for attribute–embedding correlation.

2.2.2 Correlation In addition to the size of the admissible set X_p, performance also depends on how the predicate-induced subset is arranged in the embedding space relative to the query.

To formalize the interaction between the predicate and the embedding space, define

$$g_k(q, S) = \frac{1}{k} \sum_{x \in \mathrm{NN}_k(q,S)} d(q, x),$$

where $\mathrm{NN}_k(q, S)$ denotes the k nearest neighbors of q in a set S, and d is the distance function used for retrieval. Thus, $g_k(q, X_p)$ is the average distance from q to its k nearest admissible neighbors under predicate p [17].

To determine whether these admissible neighbors are unusually close or unusually far, we compare them to a baseline in which the predicate has no geometric structure in the embedding space. Let

$$R_p \subseteq X$$

be a random subset of size $|X_p|$, sampled uniformly from X. Then R_p represents a hypothetical subset with the same cardinality as X_p, but without any predicate-specific spatial organization.

We define the query-level correlation of (q, p) as

$$C(q, p) = \mathbb{E}[g_k(q, R_p)] - g_k(q, X_p),$$

where the expectation is taken over the random draw of R_p.

This quantity admits a natural interpretation:

$$C(q, p) > 0 \quad \Longrightarrow \quad g_k(q, X_p) < \mathbb{E}[g_k(q, R_p)],$$

meaning that the admissible neighbors in X_p are, on average, closer to the query than would be expected under a random subset of the same size. In this case, the query exhibits *positive correlation*.

Conversely,

$$C(q, p) < 0 \quad \Longrightarrow \quad g_k(q, X_p) > \mathbb{E}[g_k(q, R_p)],$$

meaning that the admissible neighbors are farther from the query than expected under the random baseline. In this case, the query exhibits *negative correlation*.

Finally,

$$C(q, p) \approx 0$$

indicates *no correlation*, i.e., the predicate-induced subset behaves, from the perspective of the query, similarly to a random subset of the same size.

Intuitively, positive correlation means that the predicate selects vectors that are geometrically aligned with the query region: the valid answers tend to lie inside or near the query's local neighborhood. Negative

correlation means the opposite: although vectors satisfying the predicate exist, they are systematically displaced from the query's neighborhood, so retrieving them requires exploration beyond the region that would be most promising under vector similarity alone.

This distinction is fundamental for filtered vector search. Two predicates may have the same selectivity, i.e.,

$$|X_{p_1}| = |X_{p_2}|,$$

while inducing very different values of $C(q, p)$. In that case, the two queries have the same number of admissible vectors but differ substantially in geometric difficulty. Correlation therefore captures an aspect of filtered vector search that selectivity alone cannot express.

In this thesis, we develop a practical method to estimate this quantity efficiently.

2.3 Execution Strategies Execution strategies define how structured filter predicates are combined with nearest neighbor search. The main difference between them is when and how the filter is applied in relation to the search process. This design choice directly affects performance, efficiency, and result quality.

In this thesis, we focus on three main execution strategies.

2.3.1 Pre-filtering Let D be a dataset of vectors and ϕ a predicate. Pre-filtering first evaluates the predicate and constructs the subset

$$D_\phi = \{x \in D \mid \phi(x)\}.$$

Nearest neighbor search is then performed only over D_ϕ.

This guarantees that all returned results satisfy the predicate. However, standard vector indexes are built over the full dataset D and cannot be directly restricted to arbitrary subsets D_ϕ. In practice, this often means scanning all vectors in D_ϕ and computing distances explicitly, with cost proportional to $|D_\phi|$. When selectivity is high ($|D_\phi| \approx |D|$), the cost is close to a full scan of the dataset.

2.3.2 Post-filtering Post-filtering performs nearest neighbor search over D, ignoring the predicate. Let $C_k \subseteq D$ denote the top-k retrieved candidates. The predicate is then applied to C_k, and only elements satisfying ϕ are kept.

This allows the index to operate normally, but it does not guarantee that k valid results are obtained. If only a fraction $\text{sel}(\phi)$ of the dataset satisfies the predicate, the expected number of valid results in C_k is approximately $k \cdot \text{sel}(\phi)$. If this number is less than k, additional candidates must be retrieved, increasing the total number of distance evaluations.

2.3.3 Inline Filtering Inline filtering incorporates the predicate into the search itself, so that it directly constrains which nodes are explored. Instead of constructing D_ϕ in advance (pre-filtering) or discarding invalid results after retrieval (post-filtering), the constraint is enforced during traversal.

In graph-based indexes, this amounts to searching over the subgraph induced by the predicate, i.e., nodes $x \in D$ such that $\phi(x)$ holds. A key challenge is that this subgraph is typically not well connected, which can prevent the search from reaching relevant parts of the space.

ACORN [17] addresses this by maintaining connectivity under filtering. At query time, it begins from the standard HNSW entry point and performs greedy routing until it reaches a node satisfying the predicate. From there, the search proceeds only through nodes that satisfy the predicate, with filtering applied to neighbor expansions at each step.

To support this restricted traversal, ACORN provides two variants. ACORN-γ increases connectivity at index construction time by expanding each node's neighborhood by a factor γ, followed by pruning to control index size. This pruning is *predicate-agnostic*, i.e., based solely on standard distance-based heuristics and independent of any predicate, ensuring that the index remains general-purpose while improving connectivity for arbitrary filters. In contrast, ACORN-1 leaves the index unchanged and performs additional neighbor exploration dynamically at query time.

In this thesis, ACORN serves as the representative inline filtering method.

3. Methodology

This chapter describes the methodology used to evaluate the proposed correlation metric. The goal is to study how the relationship between structured attributes and the embedding space affects query behavior in filtered vector search.

To this end, we define a data model and a query model, and evaluate the proposed metric across different datasets and query configurations. The methodology is designed to isolate the effect of attribute–embedding interaction while remaining independent of a specific index structure.

3.1 Data Model We consider datasets of high-dimensional vectors, where each vector is associated with a structured attribute.

As a proof of concept, we use a handcrafted dataset based on SIFT1M, consisting of one million vectors of dimension 128. Structured attributes are generated and attached to these vectors to control the degree of correlation with the embedding space.

In addition, we use real-world embeddings based on GloVe to evaluate the proposed approach under more realistic data distributions.

3.2 Query Model Queries consist of a vector and a structured predicate. The predicate is defined as a range over the attribute domain.

For evaluation, we use a set of 10,000 query vectors. For each query, we retrieve the top-k nearest neighbors under the given predicate, with $k = 100$.

This setup allows us to analyze how filtering affects retrieval behavior under different query conditions.

3.3 Evaluation Setup We evaluate query performance using recall@k, comparing approximate results to exact nearest neighbors.

To isolate the effect of attribute–embedding interaction, we consider queries with varying degrees of correlation between the predicate and the embedding space. This allows us to study how the proposed metric relates to retrieval difficulty and execution behavior.

3.4 Objective

4. Dataset Construction

4.1 Design Goals Our goal is to construct synthetic structured attributes that will later be used in our experiments. We generate two types of attributes: (i) correlated attributes, which are explicitly coupled to the geometry of the vector space, and (ii) uncorrelated attributes, which serve as a random baseline.

All attributes are defined over a bounded discrete domain:

$$\mathcal{A} = \{0, \ldots, 200\}.$$

4.2 Correlated Attribute Generation We aim to enforce a strong relationship between vector similarity and attribute similarity, i.e., vectors that are close in the embedding space should receive similar attribute values.

To achieve this, we construct a smooth attribute function over the vector space. We first group vectors into clusters, assign scalar values to the cluster centroids, and then interpolate these values to all vectors using a Gaussian kernel. This results in an attribute function that is locally consistent and varies smoothly across the space.

4.2.1 Clustering Let $\{x_i\}_{i=1}^N \subset \mathbb{R}^d$ denote the set of base vectors.

We partition the dataset using MiniBatch k-means to obtain k cluster centroids:

$$\{c_j\}_{j=1}^k.$$

In practice, we choose the number of clusters proportional to the square root of the dataset size, i.e., $k \approx \sqrt{N}$, which provides a balance between representational fidelity and computational efficiency.

These centroids serve as representative points of the data distribution.

4.2.2 Cluster Anchors To each centroid c_j, we assign a scalar anchor value

$$v_j = \frac{j-1}{k-1}, \quad j \in \{1, \ldots, k\},$$

for $k > 1$, resulting in values uniformly spaced over $[0, 1]$. In the degenerate case $k = 1$, we assign $v_1 = 0$.

These anchors define the values of the attribute function at the centroid locations and act as reference values for the subsequent interpolation step.

Note that this anchor assignment is index-based rather than geometry-aware: centroid values are uniformly spaced according to centroid index, not according to their position in the embedding space. Geometric coupling is introduced later through Gaussian interpolation from the centroids to the data points.

4.2.3 Gaussian Weighting For each vector x_i, we compute its Euclidean distance to all centroids:

$$d_{ij} = \|x_i - c_j\|.$$

We then define soft assignment weights using a Gaussian kernel:

$$w_{ij} = \exp\left(-\frac{d_{ij}^2}{2\sigma^2}\right).$$

Radial Basis Function Interpretation The Gaussian kernel used in our construction is an instance of a *radial basis function* (RBF). In general, an RBF is a function that depends only on the distance between two points. Formally, it has the form

$$\phi(x, c) = g(\|x - c\|),$$

meaning that its value is determined solely by how far x is from a center c, and not by the direction.

In our case, we use the Gaussian RBF:

$$K(x_i, c_j) = \exp\left(-\frac{\|x_i - c_j\|^2}{2\sigma^2}\right).$$

This function assigns a high value when x_i is close to c_j, and a value close to zero when x_i is far away. The parameter σ controls how quickly this influence decays with distance.

Using this kernel, we will later define the continuous attribute value for each vector as

$$a_i^{(\text{cont})} = \frac{\sum_{j=1}^{k} K(x_i, c_j) v_j}{\sum_{j=1}^{k} K(x_i, c_j)}.$$

This formula can be understood intuitively as follows: the attribute value of a vector x_i is computed as a weighted average of the centroid anchor values v_j, where the weights depend on how close x_i is to each centroid. Nearby centroids contribute strongly, while distant centroids contribute very little.

This construction corresponds to a *Nadaraya–Watson kernel regression estimator*. In general, given a set of points $\{z_j\}$ with associated values $\{y_j\}$, the Nadaraya–Watson estimator defines a function

$$f(x) = \frac{\sum_j K(x, z_j)\, y_j}{\sum_j K(x, z_j)},$$

which estimates the value at x as a weighted average of nearby observations.

This construction has several important properties:

- **Smoothness:** Since the kernel is continuous, small changes in x_i lead to small changes in $a_i^{(\text{cont})}$.
- **Locality:** The attribute value is mainly influenced by nearby centroids, because distant centroids receive very small weights.
- **Geometry–metadata coupling:** If a vector lies close to a centroid c_j, then its attribute value will be close to the corresponding anchor value v_j.

4.2.4 Bandwidth Parameter The parameter $\sigma > 0$ is the *bandwidth* of the Gaussian kernel. It controls how quickly the influence of a centroid decays as a function of distance.

- If σ is small, only very close centroids receive significant weight, resulting in highly localized assignments.
- If σ is large, many centroids contribute with similar weight, leading to smoother but less discriminative assignments.

Thus, σ determines the geometric scale at which the vector space is observed and directly affects the strength of the induced correlation between vectors and attributes.

In the implementation, the bandwidth is estimated from the data and optionally scaled by a constant factor `sigma_scale`. The description here corresponds to the default setting `sigma_scale` $= 1.0$, meaning that the estimated bandwidth is used directly without additional scaling.

Bandwidth Estimation A naive computation of σ from all point-to-centroid distances $\|x_i - c_j\|$ would require $\mathcal{O}(Nk)$ distance evaluations, which is computationally expensive for large-scale datasets.

To reduce this cost, we estimate σ using a random subsample. Let $S \subset \{1, \ldots, N\}$ be a uniformly sampled subset with $|S| = \min(10^4, N)$. We define:

$$\sigma = \text{median}\left(\{\|x_i - c_j\| : i \in S,\ j \in \{1, \ldots, k\}\}\right).$$

This provides a robust and computationally efficient estimate of the characteristic distance scale.

Why the Median? We use the median instead of the mean to obtain robustness to outliers. In high-dimensional spaces, distance distributions often exhibit heavy tails, and the mean can be skewed by extreme values. The median provides a more stable estimate of the typical distance, ensuring that the kernel maintains an appropriate balance between locality and smoothness.

Monte Carlo Interpretation This estimation can be viewed as a Monte Carlo approximation of the global distance distribution. Rather than computing statistics over all Nk distances, we approximate the median using a random sample. As the sample size increases, the estimate converges to the true median of the full distribution.

4.2.5 Soft Assignment The weights w_{ij} induce a *soft assignment* of each vector to all clusters simultaneously.

In contrast to hard clustering, where each vector is assigned to a single cluster,

$$x_i \mapsto \arg\min_j \|x_i - c_j\|,$$

soft assignment produces a distribution over clusters:

$$x_i \mapsto (w_{i1}, \ldots, w_{ik}).$$

Each weight w_{ij} reflects the contribution of centroid c_j to the representation of x_i. Nearby centroids receive higher weights, while distant centroids contribute negligibly.

4.2.6 Normalization and Quantization We normalize:

$$\hat{a}_i = \frac{a_i^{(\text{cont})} - \min_i a_i^{(\text{cont})}}{\max_i a_i^{(\text{cont})} - \min_i a_i^{(\text{cont})}}.$$

If all values are identical, we set $\hat{a}_i = 0.5$ for all i.

Finally, we quantize:

$$a_i = \text{round}(200 \cdot \hat{a}_i), \quad a_i \in \{0, \ldots, 200\}.$$

4.3 Uncorrelated Attribute Generation For the uncorrelated baseline, each vector is assigned an attribute value independently and uniformly at random from $\mathcal{A}$:

$$a_i^{(\text{rand})} \sim \text{Unif}(\{0, \ldots, 200\}).$$

This construction is independent of the vector space and removes any relationship between vector similarity and attribute similarity.

4.4 Query Range Generation We describe how query-dependent attribute filter ranges are constructed under controlled selectivity and varying correlation regimes.

Let

$$\{q_t\}_{t=1}^{n_q} \subset \mathbb{R}^d$$

be the set of query vectors, and let

$$\{a_i\}_{i=1}^{N}, \quad a_i \in \{0, \ldots, 200\},$$

denote the attribute values associated with the dataset.

Given a target selectivity $s \in (0, 1]$, we define the nominal number of matching points as

$$T = \max\big(1, \min(N, \lfloor sN \rfloor)\big).$$

We further define the globally sorted attribute array

$$A_{\text{sort}} = \text{sort}(a_1, \ldots, a_N),$$

which serves as the basis for constructing attribute-value intervals.

4.4.1 Nearest-Neighbor Attribute Summary For each query q_t, we perform an exact nearest-neighbor search in the vector space and retrieve its k nearest dataset points:

$$N_t = \{i_1, \ldots, i_k\}.$$

Their corresponding attribute values are collected as

$$A_t = \{a_{i_1}, \ldots, a_{i_k}\}.$$

We define the query-specific local attribute center as the median:

$$m_t = \text{median}(A_t).$$

This quantity provides a robust estimate of the local attribute distribution around the query.

4.4.2 Positive Range Construction We first determine the insertion position of m_t in the global sorted array:

$$p_t = \text{searchsorted}(A_{\text{sort}}, m_t).$$

Let

$$h = \left\lfloor \frac{T}{2} \right\rfloor.$$

We define the clipped rank interval:

$$\ell_t = \max(0,\, p_t - h), \qquad u_t = \min(N-1,\, p_t + h).$$

The resulting attribute filter is

$$[L_t^+, U_t^+] = [A_{\text{sort}}[\ell_t],\ A_{\text{sort}}[u_t]].$$

Remark (Selectivity). This construction targets a window of size approximately T in rank space. However, since filtering is performed over attribute *values* rather than exact ranks, the number of selected points is not guaranteed to be exactly T. Due to duplicate attribute values, the realized selectivity may deviate from the target.

4.4.3 Negative Range Construction We first compute the global median:

$$m_g = \text{median}(A_{\text{sort}}).$$

We define two global tail intervals:

$$[L_{\text{left}}, U_{\text{left}}] = [A_{\text{sort}}[0],\ A_{\text{sort}}[\min(T-1,\, N-1)]],$$
$$[L_{\text{right}}, U_{\text{right}}] = [A_{\text{sort}}[\max(0,\, N-T)],\ A_{\text{sort}}[N-1]].$$

For each query, the negative range is defined as:

$$[L_t^-, U_t^-] = \begin{cases} [L_{\text{left}}, U_{\text{left}}], & \text{if } m_t > m_g, \\ [L_{\text{right}}, U_{\text{right}}], & \text{otherwise.} \end{cases}$$

Note that the "otherwise" case includes $m_t = m_g$, in which case the right-tail interval is selected.

Interpretation. Queries whose local neighborhood contains high attribute values are paired with low-valued ranges, and vice versa. This induces systematic anti-correlation between vector similarity and attribute filtering.

4.4.4 Random Range Construction For the random baseline, attribute ranges are generated independently of the query vectors.

We sample a starting index

$$r_t \sim \text{Uniform}\{0, \ldots, N-T\},$$

and define the interval

$$[L_t^{(r)}, U_t^{(r)}] = [A_{\text{sort}}[r_t],\ A_{\text{sort}}[\min(N-1,\, r_t + T)]].$$

This produces a contiguous interval in sorted value space with nominal width corresponding to T ranks.

4.4.5 Selectivity Control Given a filter range $[L, U]$, the realized selectivity is

$$\mathrm{sel}(L, U) = \frac{1}{N} \left| \{ i \in \{1, \ldots, N\} \mid L \leq a_i \leq U \} \right|.$$

In practice, this is computed using binary search on the sorted array:

$$\begin{aligned} \text{left} &= \mathrm{searchsorted}(A_{\mathrm{sort}}, L, \text{left}), \\ \text{right} &= \mathrm{searchsorted}(A_{\mathrm{sort}}, U, \text{right}), \\ \mathrm{sel}(L, U) &= \frac{\text{right} - \text{left}}{N}. \end{aligned}$$

Key Observation. Selectivity is *controlled* approximately through rank-space window construction, but *realized* exactly with respect to the induced value interval.

4.5 Dataset Validation

4.5.1 Neighbor Consistency and Attribute Distribution A central property of the correlated attribute construction is that nearby vectors in the embedding space should receive similar attribute values.

To validate this, for each vector x_i, we compute its k-nearest neighbors and denote their indices by N_i. We then define the average attribute value of its neighbors as

$$\bar{a}_i = \frac{1}{|N_i|} \sum_{j \in N_i} a_j.$$

We assess local consistency by comparing a_i with $\bar{a}_i$. To visualize this, we plot each point as $(a_i, \bar{a}_i)$. Points that lie close to the diagonal indicate that a vector's attribute value agrees with the average of its neighbors.

Figure 4.1 and Figure 4.2 illustrate the results for correlated and random attributes, respectively.

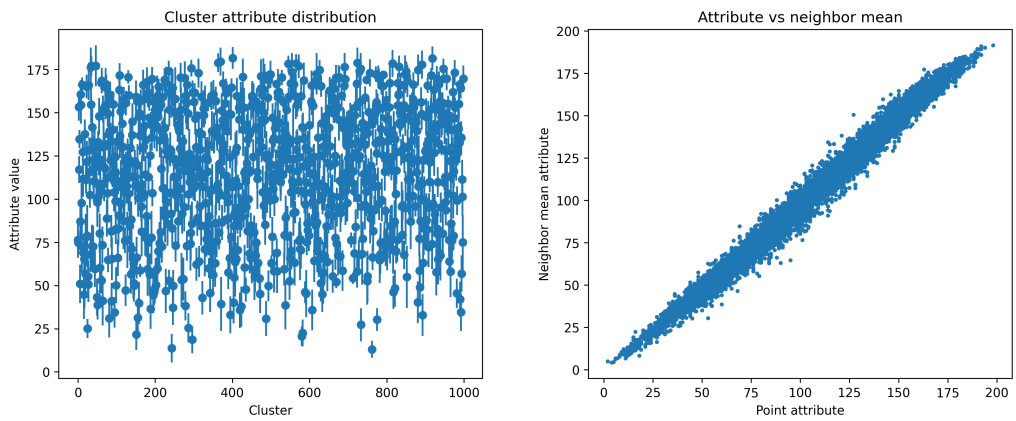

Figure 4.1: Correlated attribute. Strong diagonal trend indicates high local consistency between a_i and $\bar{a}_i$. Values span the full domain.

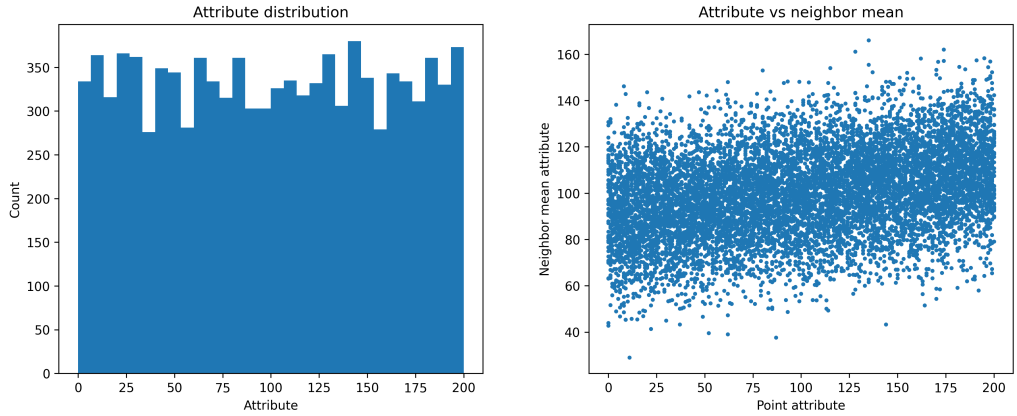

Figure 4.2: Random attribute. Points are scattered, indicating no geometric correlation.

4.5.2 PCA Visualization We project the 128-dimensional vectors onto their first principal components and color each point by its attribute value a_i, since we cannot visualize the original space. This lets us check whether nearby points have similar a_i.

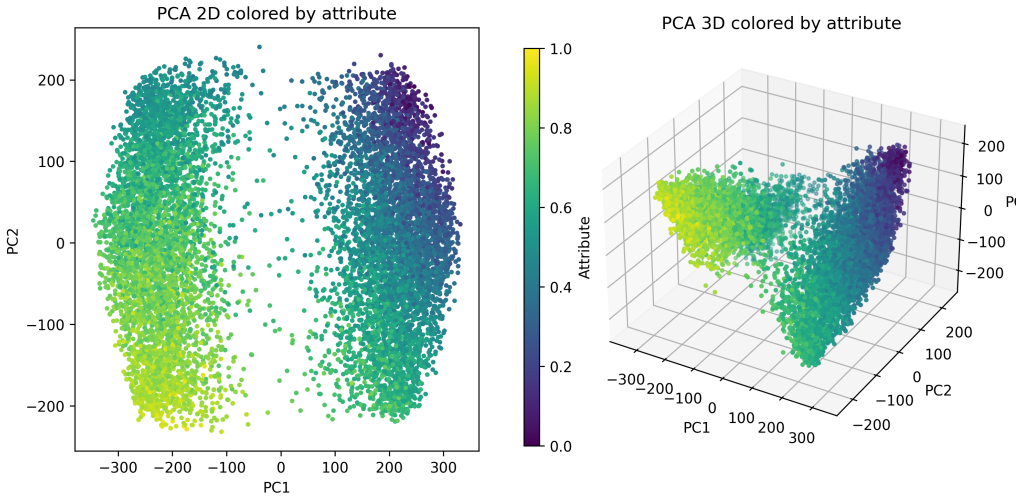

Figure 4.3: Correlated attribute. Smooth color gradients reflect alignment with the embedding geometry.

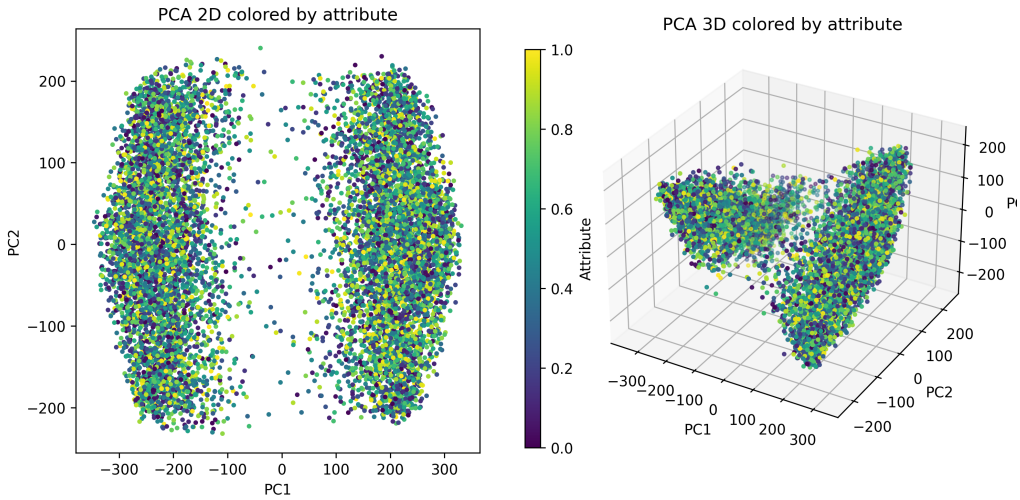

Figure 4.4: Random attribute. No visible structure; colors appear uncorrelated with the embedding.

5. Correlation Metric

Our metric captures two aspects simultaneously: (i) how different the query-local attribute distribution is from the global dataset distribution, and (ii) whether the queried filter range is locally enriched or depleted.

Formally, let $X = \{x_1, \ldots, x_N\} \subseteq \mathbb{R}^d$ be the dataset of embedding vectors, and let each vector x_i be associated with a scalar attribute a_i. Given a query vector $q \in \mathbb{R}^d$ and a filter range $[L, U]$, we seek to determine whether values in $[L, U]$ are more likely in the neighborhood of q than in the dataset as a whole.

5.1 Precomputed statistics We precompute three components:

5.1.1 Cluster centroids. Let $\ell_i \in \{1, \ldots, k\}$ denote the cluster assignment of vector x_i. For each cluster c, define

$$C_c = \{i \mid \ell_i = c\}.$$

The centroid is reconstructed as

$$c_c = \frac{1}{|C_c|} \sum_{i \in C_c} x_i.$$

5.1.2 Attribute distributions. We estimate both global and cluster-local attribute distributions using histogram-based density estimation over the scalar attribute a_i.

Bin construction. The number of histogram bins M is determined adaptively from the data. We begin by estimating the bin width using the Freedman–Diaconis rule,

$$h = \frac{2 \cdot \mathrm{IQR}}{n^{1/3}},$$

where IQR is the interquartile range of the attribute values and n is the number of samples. Given the data range $[v_{\min}, v_{\max}]$, this yields the bin-count estimate

$$\widehat{M} = \left\lceil \frac{v_{\max} - v_{\min}}{h} \right\rceil.$$

When the interquartile range is too small (i.e., the data is nearly constant), this estimate becomes unreliable, and we instead fall back to a heuristic based on a target number of samples per bin.

To keep the number of bins reasonable, we force it to stay within a fixed range:

$$M = \min\bigl(M_{\max},\, \max(M_{\min},\, \widehat{M})\bigr).$$

This way, the histogram is neither too rough (losing detail) nor too detailed (with many empty bins that make the results unreliable).

In our implementation, we fix $M_{\min} = 20$ and $M_{\max} = 200$. Since the dataset characteristics are known, these bounds are sufficient and yield stable results in practice.

More generally, these bounds can be defined in a data-dependent manner. For example, letting u denote the number of distinct attribute values, one may use

$$M_{\min} = \min\Big(u,\, \max\Big(5,\, \Big\lfloor n^{1/3} \Big\rfloor\Big)\Big), \qquad M_{\max} = \min\Big(u,\, \Big\lfloor \frac{n}{m_{\min}} \Big\rfloor\Big),$$

where $m_{\min}$ is the minimum desired number of samples per bin. Such choices adapt the binning resolution to both dataset size and attribute cardinality.

Global distribution. Given bin edges $b_0 < b_1 < \cdots < b_M$, we first compute a histogram over the full dataset:

$$H_{\text{global}}(j) = |\{\, i \mid a_i \in [b_j, b_{j+1}) \,\}|, \qquad j = 0, \ldots, M-1.$$

We then convert this histogram into a probability distribution. To avoid zero probabilities, we apply additive smoothing by adding a small constant $\varepsilon > 0$ to each bin:

$$P_{\text{global}}(j) = \frac{H_{\text{global}}(j) + \varepsilon}{\sum_{k=0}^{M-1} \big(H_{\text{global}}(k) + \varepsilon\big)}.$$

This ensures that no bin has zero probability. Without smoothing, bins with zero counts would lead to undefined or unstable values in downstream computations (e.g., divisions or logarithms in divergence measures).

After smoothing, the histogram no longer sums to one, so we renormalize it. We simply rescale all values so that they form a valid probability distribution again:

$$\sum_j P_{\text{global}}(j) = 1.$$

The corresponding cumulative distribution function is

$$\text{CDF}_{\text{global}}(j) = \sum_{k=0}^{j} P_{\text{global}}(k).$$

Cluster-local distributions. For each cluster c, we first compute its radius

$$r_c = \max_{i \in C_c} \|x_i - c_c\|_2.$$

We then introduce a **scaling parameter** $\lambda \geq 1$ to slightly enlarge the cluster and define the set of selected points

$$C'_c = \{\, i \in C_c \mid \|x_i - c_c\|_2 \leq \lambda r_c \,\}.$$

This allows us to include points near the boundary of the cluster, making the estimate more robust. The choice and effect of λ are discussed in Section 5.2.3.2.

Using these points, we estimate the cluster-level attribute distribution by constructing a histogram over the same bins:

$$H_c(j) = |\{\, i \in C'_c \mid a_i \in [b_j, b_{j+1}) \,\}|, \qquad j = 0, \ldots, M-1,$$

which corresponds to the empirical distribution

$$P_c(a) = \frac{1}{|C'_c|} \sum_{i \in C'_c} \mathbf{1}[a_i \in \text{bin}(a)].$$

We then convert it into a probability distribution by applying additive smoothing and renormalization, as in the global case:

$$P_c(j) = \frac{H_c(j) + \varepsilon}{\sum_{k=0}^{M-1} \big(H_c(k) + \varepsilon\big)}.$$

Finally, the cumulative distribution function is computed analogously:

$$\mathrm{CDF}_c(j) = \sum_{k=0}^{j} P_c(k).$$

Query-conditional distribution. We consider the *query-conditional attribute distribution* $P(a \mid x)$, which describes the distribution of attribute values in the neighborhood of a point $x \in \mathbb{R}^d$. Intuitively, it captures how attribute values are distributed among vectors that are similar to x in the embedding space.

Estimating $P(a \mid x)$ directly is challenging, as it requires defining a local neighborhood around each query and computing a distribution from a potentially small number of samples. Instead, we approximate the mapping

$$x \mapsto P(a \mid x)$$

by assigning a single distribution $P_c(a)$ to each cluster:

$$x \mapsto P_c(a), \quad \text{for } x \in R_c,$$

where R_c denotes the region associated with cluster c. This avoids estimating a separate distribution for every query location, which would be both computationally expensive and statistically unreliable.

Distributions and estimation. We distinguish between three distributions.

The conditional distribution $P(a \mid x)$ is defined at the point level.

At the cluster level, we introduce an ideal (population-level) distribution $P_c(a)$, which we define as the average of the conditional distributions over the region R_c:

$$P_c(a) = \mathbb{E}_{x' \sim \mathrm{Unif}(R_c)}[P(a \mid x')].$$

This provides a clean abstraction of the attribute distribution associated with the cluster.

In practice, this distribution is not directly observable and is instead estimated empirically using a histogram over the points in the cluster:

$$\hat{P}_c(a) = \frac{1}{n_c} \sum_{i \in C'_c} \mathbf{1}[a_i \in \mathrm{bin}(a)].$$

5.1.3 Query-to-cluster mapping. Given a query $q \in \mathbb{R}^d$, we associate it with a representative region of the dataset by assigning it to its nearest centroid. To this end, we construct an index over the set of centroids $\{c_1, \ldots, c_k\}$ and define

$$c^*(q) = \arg\min_c \|q - c_c\|_2.$$

This induces a partition of the vector space into Voronoi regions and assigns each query to the cluster whose centroid is closest to it.

To estimate the attribute distribution in the vicinity of q, we use the empirical distribution of its assigned cluster, given by the histogram-based estimate computed in the previous section.

$$P(a \mid q) \approx P_{c^*(q)}(a),$$

where $P_{c^*(q)}$ denotes the attribute distribution associated with the cluster $c^*(q)$.

5.2 Approximation quality We now analyze how well the cluster-based approximation

$$P(a \mid q) \approx P_c(a)$$

captures the true query-conditional distribution.

Smoothness assumption We assume that attribute distributions are correlated with the underlying vector space geometry, meaning that nearby points exhibit similar attribute behavior.

Formally, we assume a *Lipschitz continuity* condition: there exists a constant $L > 0$ such that

$$D(P(a \mid x), P(a \mid x')) \leq L\|x - x'\|$$

for all $x, x' \in \mathbb{R}^d$, where $D(\cdot, \cdot)$ measures the difference between two distributions.

This means that the change in the attribute distribution is proportional to the distance between points: small movements in the embedding space lead to small changes in the distribution, while large differences can only occur between points that are far apart.

5.2.1 Bounding the approximation error For a point $x \in R_c$, the approximation error is

$$D(P(a \mid x), P_c(a)),$$

where $D(\cdot, \cdot)$ is a divergence between distributions.

Recall that we define the ideal cluster-level distribution as the average of the conditional distributions over the region R_c, i.e., $P_c(a)$ is the average of $\{P(a \mid x') : x' \in R_c\}$.

We now bound the deviation between $P(a \mid x)$ and this average. Since $P_c(a)$ is an average over these distributions, its distance to $P(a \mid x)$ can be bounded by the largest deviation within the cluster:

$$D(P(a \mid x), P_c(a)) \leq \sup_{x' \in R_c} D(P(a \mid x), P(a \mid x')).$$

Applying the smoothness assumption gives:

$$D(P(a \mid x), P_c(a)) \leq \sup_{x' \in R_c} D(P(a \mid x), P(a \mid x'))$$

$$D(P(a \mid x), P_c(a)) \leq \sup_{x' \in R_c} L\|x - x'\|$$

$$D(P(a \mid x), P_c(a)) \leq L \sup_{x' \in R_c} \|x - x'\|.$$

Taking the supremum over all $x \in R_c$ yields:

$$\sup_{x \in R_c} D(P(a \mid x), P_c(a)) \leq L \sup_{x, x' \in R_c} \|x - x'\|.$$

Cluster diameter. We define the diameter of a cluster R_c as

$$\operatorname{diam}(R_c) = \sup_{x,x' \in R_c} \|x - x'\|.$$

Substituting this into the previous bound yields

$$\sup_{x \in R_c} D(P(a \mid x), P_c(a)) \leq L \operatorname{diam}(R_c).$$

5.2.2 Error decomposition The proposed approximation introduces two main sources of error.

The first is the quantization (approximation) error,

$$P(a \mid x) \ \text{ vs } \ P_c(a),$$

which arises from replacing the pointwise conditional distribution with a cluster-level approximation.

The second is the estimation error,

$$P_c(a) \ \text{ vs } \ \hat{P}_c(a),$$

which is introduced when estimating the cluster-level distribution from a finite number of samples using a histogram.

5.2.2.1 Quantization error (Geometric Discretization). For a query assigned to cluster c, we define the worst-case approximation error as

$$\varepsilon_{\text{quant}}(q) = \sup_{x \in R_c} D(P(a \mid x), P_c(a)),$$

where $D(\cdot, \cdot)$ is a divergence between distributions (e.g., total variation or Jensen–Shannon). This measures the error introduced by replacing the pointwise distribution $P(a \mid x)$ with a single cluster-level distribution.

From the previous bound, this error satisfies

$$\varepsilon_{\text{quant}}(q) \leq L \operatorname{diam}(R_c),$$

where $\operatorname{diam}(R_c)$ is the cluster diameter and L is the smoothness constant. Thus, the approximation error increases with the size of the cluster and with how rapidly the distribution changes in the embedding space.

However, this bound is worst-case and does not fully reflect practical behavior. In particular, the error also depends on how the geometry of the cluster aligns with the structure of $P(a \mid x)$.

Refinements beyond diameter. Three factors are particularly important:

Cluster anisotropy. The diameter only captures the maximum pairwise distance, but ignores the shape of the cluster. As a result, clusters can be anisotropic (e.g., elongated or skewed).

Even if most points are close to the centroid, an elongated cluster can still contain points that are far apart. Along the short axis, points are very similar, while along the long axis, points may differ significantly. Thus, two points in the same cluster can have very different attribute distributions despite being assigned the same $P_c(a)$.

Alignment with distribution gradients. Think of $P(a \mid x)$ as assigning, to every point x in the embedding space, a full distribution over attribute values:

$$x \mapsto P(a \mid x).$$

If we fix a location x and look at vectors near x, the attribute values of those vectors follow a distribution $P(a \mid x)$. Moving to a different point x' may change this distribution. Thus, as we move through the embedding space, we can think of the attribute distribution as evolving over space.

Case 1: Good alignment (low error). If a cluster lies within a region where this distribution changes little, then for any two points $x, x' \in R_c$, we have $P(a \mid x) \approx P(a \mid x')$. In this case, all points in the cluster exhibit similar attribute behavior, and using a single distribution $P_c(a)$ is a good approximation.

Case 2: Poor alignment (high error). If a cluster spans a region where the distribution changes significantly, then points within the cluster may have very different attribute distributions. However, the model still assigns a single $P_c(a)$, which forces these different behaviors to be averaged together and introduces systematic bias.

For example, suppose that in one region of the space the attributes are concentrated around 10, while in another region they are concentrated around 100. If a cluster spans both regions, then $P_c(a)$ mixes these two distinct distributions and fails to accurately represent either.

Local curvature. Finally, the smoothness assumption only captures first-order (linear) variation. If $P(a \mid x)$ varies nonlinearly, the Lipschitz bound may underestimate the true variation within the cluster. As a result, even clusters with small diameter can incur non-negligible approximation error.

5.2.2.2 Estimation error (Statistical Estimation). Recall the ideal (population-level) definition of the cluster distribution:

$$P_c(a) = \mathbb{E}_{x' \in R_c}[P(a \mid x')].$$

This represents the true cluster-level distribution, which is not directly observable.

In practice, we estimate this distribution empirically using a histogram over the points in the cluster:

$$\hat{P}_c(a) = \frac{1}{n_c} \sum_{i \in C'_c} \mathbf{1}[a_i \in \text{bin}(a)],$$

where $n_c = |C'_c|$ is the number of samples in the cluster. This estimator relies on discretizing the attribute values into bins.

The estimation error is defined as

$$\varepsilon_{\text{est}}(c) = D(\hat{P}_c(a), P_c(a)),$$

which measures the deviation between the empirical estimate and the true cluster-level distribution. This error arises from several sources:

Finite sampling noise (variance). We estimate the distribution using a finite number of samples, which introduces sampling noise that decreases with the sample size:

$$\varepsilon_{\text{est}}(c) \sim \mathcal{O}\left(\frac{1}{\sqrt{n_c}}\right).$$

This scaling corresponds to the standard deviation of empirical estimates. Thus, large clusters yield more stable estimates, while small clusters produce noisy distributions.

Histogram discretization (bias). We do not estimate the full distribution, but a binned version $P_c(a) \approx P_c(\text{bin}(a))$. As a result, fine structure within bins is lost and values near bin boundaries may be distorted. This introduces systematic bias, as the resolution of the distribution is reduced.

Interaction with cluster size. Small clusters have few samples (n_c), leading to high variance and unstable estimates of $P_c(a)$. Large clusters have more samples and therefore lower variance, but tend to cover more heterogeneous regions, which increases the approximation error.

5.2.3 Bias–variance trade-off The proposed method introduces a trade-off between approximation (quantization) error and estimation error. This trade-off is controlled by two parameters: the number of clusters k and the radius expansion parameter λ.

5.2.3.1 Effect of the number of clusters k. The number of clusters k determines the granularity of the partition of the embedding space.

Increasing k reduces the cluster diameter, leading to lower quantization error, but also reduces the number of samples per cluster, resulting in higher estimation error.

Decreasing k increases the cluster diameter, leading to higher quantization error, but increases the number of samples per cluster, resulting in lower estimation error.

5.2.3.2 Effect of radius expansion λ. In addition to clustering, we can expand each cluster region using a parameter $\lambda > 1$. We define the expanded region

$$r'_c = \lambda\, r_c, \quad R'_c = \{x : \|x - c_c\| \leq r'_c\}.$$

The diameter of the expanded region satisfies

$$\mathrm{diam}(R'_c) = \sup_{x,x' \in R'_c} \|x - x'\|.$$

Using the triangle inequality,

$$\|x - x'\| \leq \|x - c_c\| + \|x' - c_c\| \leq r'_c + r'_c = 2r'_c,$$

which implies

$$\mathrm{diam}(R'_c) \leq 2r'_c = 2\lambda r_c.$$

Thus, λ directly controls an upper bound on the cluster diameter, and consequently the quantization error.

Increasing λ enlarges the effective region R'_c, increasing the diameter (and thus the potential quantization error), while also increasing the number of samples n_c, which reduces estimation variance. Decreasing λ has the opposite effect: it reduces the diameter and quantization error, but also reduces the sample size, increasing estimation variance.

Overall, λ acts as a regularization parameter, trading off geometric fidelity against statistical stability by controlling the spatial extent over which the conditional distribution is assumed constant.

6. System & Execution Strategies

6.1 Pre-filtering

6.2 Post-filtering

6.3 Inline Filtering

6.4 Implementation Details

7. Experimental Setup

7.1 Offline computed statistics

7.2 Evaluation Metrics

7.3 Ground Truth Computation

8. Results & Discussion

Bibliography

[1] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, *Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints*, 2022. arXiv: 2203.13601 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2203.13601

[2] J. Wang et al., "Milvus: A purpose-built vector data management system," in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD '21, Virtual Event, China: Association for Computing Machinery, 2021, 2614–2627, ISBN: 9781450383431. DOI: 10.1145/3448016.3457550 [Online]. Available: https://doi.org/10.1145/3448016.3457550

[3] X. Xu, C. Li, Y. Wang, and Y. Xia, "Multiattribute approximate nearest neighbor search based on navigable small world graph," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 24, e5970, 2020. DOI: https://doi.org/10.1002/cpe.5970 eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.5970. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.5970

[4] S. Gollapudi et al., "Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters," in *Proceedings of the ACM Web Conference 2023*, ser. WWW '23, Austin, TX, USA: Association for Computing Machinery, 2023, 3406–3416, ISBN: 9781450394161. DOI: 10.1145/3543507.3583552 [Online]. Available: https://doi.org/10.1145/3543507.3583552

[5] F. Almeida and G. Xexéo, *Word embeddings: A survey*, 2023. arXiv: 1901.09069 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1901.09069

[6] L. Ma et al., *A comprehensive survey on vector database: Storage and retrieval technique, challenge*, 2026. arXiv: 2310.11703 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2310.11703

[7] S. Arya, D. M. Mount, N. S. Netanyahu, R. Silverman, and A. Y. Wu, "An optimal algorithm for approximate nearest neighbor searching fixed dimensions," *J. ACM*, vol. 45, no. 6, 891–923, Nov. 1998, ISSN: 0004-5411. DOI: 10.1145/293347.293348 [Online]. Available: https://doi.org/10.1145/293347.293348

[8] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, ser. STOC '98, Dallas, Texas, USA: Association for Computing Machinery, 1998, 604–613, ISBN: 0897919629. DOI: 10.1145/276698.276876 [Online]. Available: https://doi.org/10.1145/276698.276876

[9] O. Jafari, P. Maurya, P. Nagarkar, K. M. Islam, and C. Crushev, *A survey on locality sensitive hashing algorithms and their applications*, 2021. arXiv: 2102.08942 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2102.08942

[10] A. Gionis, P. Indyk, and R. Motwani, "Similarity search in high dimensions via hashing," in *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, Edinburgh, Scotland, 1999, pp. 518–529. [Online]. Available: https://www.cs.princeton.edu/courses/archive/spring13/cos598C/Gionis.pdf

[11] Y. A. Malkov and D. A. Yashunin, *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs*, 2018. arXiv: 1603.09320 [cs.DS]. [Online]. Available: https://arxiv.org/abs/1603.09320

[12] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32, Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf

[13] O. Salemohamed, L. Charlin, S. Garg, V. Sharan, and G. Valiant, *Discovering data structures: Nearest neighbor search and beyond*, 2024. arXiv: 2411.03253 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2411.03253

[14] C.-Y. Chiu, A. Prayoonwong, and Y.-C. Liao, *Learning to index for nearest neighbor search*, 2019. arXiv: 1807.02962 [cs.IR]. [Online]. Available: https://arxiv.org/abs/1807.02962

[15] M. Douze et al., *The faiss library*, 2025. arXiv: 2401.08281 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2401.08281

[16] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with gpus*, 2017. arXiv: 1702.08734 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1702.08734

[17] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, *Acorn: Performant and predicate-agnostic search over vector embeddings and structured data*, 2024. arXiv: 2403.04871 [cs.IR]. [Online]. Available: https://arxiv.org/abs/2403.04871

[18] Y. Cai, J. Shi, Y. Chen, and W. Zheng, "Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, Dec. 2024. DOI: 10.1145/3698822 [Online]. Available: https://doi.org/10.1145/3698822

[19] Y. Chronis, H. Caminal, Y. Papakonstantinou, F. Özcan, and A. Ailamaki, "Filtered vector search: State-of-the-art and research opportunities," *Proc. VLDB Endow.*, vol. 18, no. 12, 5488–5492, Aug. 2025, ISSN: 2150-8097. DOI: 10.14778/3750601.3750700 [Online]. Available: https://doi.org/10.14778/3750601.3750700

[20] L. Getoor, B. Taskar, and D. Koller, "Selectivity estimation using probabilistic models," *SIGMOD Rec.*, vol. 30, no. 2, 461–472, May 2001, ISSN: 0163-5808. DOI: 10.1145/376284.375727 [Online]. Available: https://doi.org/10.1145/376284.375727

[21] C. A. Lynch, "Selectivity estimation and query optimization in large databases with highly skewed distribution of column values," in *Very Large Data Bases Conference*, 1988. [Online]. Available: https://api.semanticscholar.org/CorpusID:20418479

[22] H. Zhang, C. Mayer, M. A. Khamis, D. Olteanu, and D. Suciu, *Lpbound: Pessimistic cardinality estimation using ℓ_p-norms of degree sequences*, 2025. arXiv: 2502.05912 [cs.DB]. [Online]. Available: https://arxiv.org/abs/2502.05912

A. First Appendix

B. Second Appendix